

OPTIMIZING WASTEWATER TREATMENT THROUGH AI

A Project Report Submitted to the Madurai Kamaraj University in

Partial Fulfillment for the Award of the Degree

BACHELOR OF COMPUTER APPLICATIONS

(2022 - 2025)

Submitted by

SOMU R

22BCA175

Under the Guidance of

Mrs. S.S.SARMILA M.C.A.,M.E.,



THE AMERICAN COLLEGE

An Autonomous Institution affiliated to Madurai Kamaraj University

(Re-accredited (3rd cycle) by NAAC with Grade “A+” CGPA-3.47 on a 4-points scale)

Madurai – 625002.

DECLARATION

I hereby declare that this project was carried out by me under the guidance of **Mrs. S.S.SARMILA M.C.A.,M.E.**, Department of Computer Applications, The American College, Madurai.

I also declare that this project report is the result of my own effort and that it has not been copied from any one and has not been submitted by anybody, anywhere.

Place: Madurai

Date:

SOMU R

(22BCA175)



Since 1881

THE AMERICAN COLLEGE

DEPARTMENT OF COMPUTER APPLICATIONS

An Autonomous Institution affiliated to Madurai Kamaraj University

(Re-accredited (3rd cycle) by NAAC with Grade “A+” CGPA-3.47 on a 4-points scale)

Madurai – 625002

CERTIFICATE

This is to certify that this Project titled **OPTIMIZING WASTEWATER TREATMENT THROUGH ARTIFICIAL INTELLIGENCE** is a bonafide work done by **SOMU R (22BCA175)** in Partial Fulfillment for the award of the degree of **Bachelor of Computer Applications** of **The American College, Madurai** for the year 2022-2025.

Mrs. S.S.SARMILA M.C.A.,M.E.,

Internal Guide

Mrs. T. Suganya, M.Sc., M.Phil., SLET, (Ph.D)

Head of the Department

Submitted for the Viva-Voice examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

I am pleased to convey my sincere thanks to our Principal **Dr. M. Davamani Christofer, M.Sc., M.Phil., M.Ed., PGDCA, Ph.D.(Inter Disciplinary), Ph.D.(Math).**

I am Grateful to the Head of our department **Mrs. T. Suganya, M.Sc., M.Phil., SLET, (Ph.D)** for her energetic encouragement and support.

I express my deep sense of gratitude and indebtedness to **Mrs. S.S.SARMILA M.C.A.,M.E.,** his/her inspiring guidance and continuous encouragement which has enabled me to complete this project.

Also I would like to thank my Teachers, Friends and my well wishers who have helped me during the course of completion of this project.

Contents

S. NO.	DESCRIPTION	PAGE NO.
	Abstract	
1	System Analysis 1.1 Existing System 1.2 Proposed System 1.3 Feasibility Study	1 2 3 4
2	System Requirements Specification 2.1 Software Requirements 2.2 Hardware Requirements 2.3 About Software	7 8 9
3	System Design 3.1 Modules Description 3.2 Functional Requirements 3.2.1 System Diagram 3.2.2 Data Flow Diagrams 3.3 User Interface Design	13 15 17 18 19
4	System Implementation	22
5	System Testing	23
6	System Maintenance	25
7	Conclusion	27
8	Future Enhancement	28
9	Bibliography	29
10	Appendix 10.1 Sample Screens 10.2 Source Code	30 32

ABSTRACT

Artificial Intelligence (AI) is increasingly being applied in wastewater treatment for process optimization which involves the use of AI algorithms for the real-time analysis of data from sensors to optimize the various stages of wastewater treatment. This includes monitoring parameters such as pH, and DO levels, turbidity, tds, and temperature. This includes detecting unusual variations in water quality parameters that may indicate system malfunctions or the presence of pollutants.

The purpose of this work was to use machine learning techniques such as SVM, RF and XG Boost method to categorize a dataset of water quality. Water quality is dictated by features such as dissolved oxygen (DO), temperature, pH, Turbidity and Total Dissolved Sensor(TGS). These features are handled in five steps: data pre-processing using min-max normalization and missing data management using RF, feature correlation, applied machine learning classification, and models feature importance.

1.SYSTEM ANALYSIS

The primary objective of this project is to introduce an intelligent water quality monitoring system in IoT (Internet of Things) platform which would help to monitoring different physical parameters of the Wastewater rather than relying on manual process. Moreover, IoT is a system of alliance among various devices and the competence of deportation data over the system.

The major goal of this study is to develop a computationally efficient and reliable method aimed at estimating water quality characteristics to reduce the quantity of our time and money during measuring them.

A dataset with pH and turbidity water quality metrics, along with their accompanying water quality labels, can be used to train using Extreme Gradient Boost model, comprising of Support Vector Machine and Random Forest. Based on fresh pH and turbidity measurements, the model will then be able to monitor water quality. Since they can manage non-linear correlations between the input characteristics & the output labels that are less prone to overfitting than other tree-based models

1.1. EXISTING SYSTEM

Water scarcity drives our focus towards achieving maximum resource recovery. Zero waste generation is one of the ideal pathways towards achieving a circular economy, which brings remarkable transformation of wastewater treatment systems through commercialization by adding value management processes (Matheri et al. 2022).

To ease the complexity of wastewater treatment systems, artificial intelligence and machine learning algorithms are used to improve the intelligent systems and manage complex dynamics of mathematical models to effectively optimize the operational conditions of the wastewater treatment systems (Oruganti et al. 2023)

Currently, artificial intelligence and machine learning algorithms have been widely integrated into the existing operational management system of wastewater treatment plants, improving the water quality monitoring system (Chawishborwornworng et al. 2023)

DISADVANTAGES OF EXISTING SYSTEM.

The speed of artificial intelligence algorithms affects the data processing power when it comes to accessing the data in real time to perform analysis and facilitate decision-making processes. Slow processing power and prolonged latency lead to undesirable consequences, resulting in delayed project timeline.

Minimization of energy consumption is an important agenda when integrating compatible hardware with existing systems to deploy artificial intelligence technology. New integration systems with optimization modes to reduce power consumption would ease the financial burden on the business and wastewater treatment industry.

Artificial intelligence systems require enormous computational energy to process and analyse data. Computational power grows immensely as the data grows, requiring algorithms to manage the voluminous data and minimize power consumption.

1.2. PROPOSED SYSTEM

For our research, we have used a collection of three distinct models to compare the accuracy of our model to that of other current water quality prediction systems, we have focused on accuracy measurement. For our prediction, we have applied numerous machine learning algorithms like SVM, RF and XG Boost method.

We can handle and analyse large datasets and requires less processing power. In addition, complex, nonlinear variables of wastewater quality parameters can be modelled using computed autoregressive integrated moving average to forecast the levels of nitrogen, biochemical oxygen demand, chemical oxygen demand, phosphorus, ammonia, total suspended solids with relatively high accuracy ranging between 71 and 97% for the training data and low prediction errors less than 9% for the testing data

ADVANTAGES OF PROPOSED SYSTEM

Integration of IoT sensors for real-time data collection (e.g., pH, tds, temperature, turbidity, dissolved oxygen).

Use of AI/ML algorithms for: Process optimization, Predictive maintenance of equipment, Real-time monitoring and anomaly detection.

1.3. FEASIBILITY STUDY

A feasibility study is crucial to determine the practicality and viability of implementing AI in a wastewater treatment plant. It encompasses technical, economic, and operational aspects.

Technical Feasibility

➤ **Availability of Technology:**

- AI and ML algorithms suitable for wastewater treatment are readily available, either as open-source libraries or commercial software packages.
- IoT sensors for real-time monitoring are mature and widely used in various industries.
- Cloud computing platforms provide scalable infrastructure for data storage and processing.

➤ **Integration with Existing Infrastructure:**

- The AI system should be designed to integrate seamlessly with existing SCADA systems and control infrastructure.
- Retrofitting sensors and control devices into existing plants may require some modifications, but is generally feasible.
- Interoperability standards (e.g., Modbus, OPC) facilitate communication between different components.

➤ **Data Availability and Quality:**

- Sufficient historical data is required to train AI models effectively.
- Data quality is critical; sensors must be calibrated regularly, and data cleaning procedures should be implemented.
- Data augmentation techniques can be used to supplement limited datasets.

➤ **Accuracy and Reliability:**

- The accuracy of AI predictions must be validated through rigorous testing and evaluation.
- The system should be designed to handle missing or erroneous data gracefully.
- Redundancy and fail-safe mechanisms should be implemented to ensure reliable operation.

Economic Feasibility

➤ **Capital Costs:**

- Initial investment in sensors, AI software, computing infrastructure, and system integration.
- Cost of training and consulting services.

➤ **Operational Costs:**

- Ongoing costs for data storage, software licenses, and maintenance.
- Potential reduction in energy and chemical consumption.
- Reduced labor costs due to automation.

➤ **Return on Investment (ROI):**

- Quantifiable benefits include reduced energy costs, chemical usage, maintenance expenses, and improved effluent quality.
- Cost-benefit analysis to determine the payback period and overall ROI.
- Consideration of potential revenue streams from resource recovery (e.g., biogas production).

➤ **Funding Sources:**

- Explore potential funding opportunities through government grants, subsidies, and private investment.

Operational Feasibility

➤ Ease of Use:

- The AI system should have a user-friendly interface that is easy for plant operators to understand and use.
- Training programs should be provided to ensure that operators are proficient in using the system.

➤ Impact on Plant Operations:

- The AI system should be designed to minimize disruptions to existing plant operations.
- Phased implementation may be necessary to gradually integrate the AI system into the plant.

➤ Regulatory Compliance:

- The AI system should be designed to ensure compliance with all relevant environmental regulations and discharge limits.
- The system should provide documentation and reporting capabilities to facilitate regulatory compliance.

➤ Scalability:

- The AI system should be scalable to accommodate future growth and changes in plant operations.

➤ Security:

- Implement security measures to protect the system from cyber threats and unauthorized access.
- Ensure data privacy and compliance with relevant data protection regulations.

2. SYSTEM REQUIREMENTS SPECIFICATION

2.1. SOFTWARE REQUIREMENTS

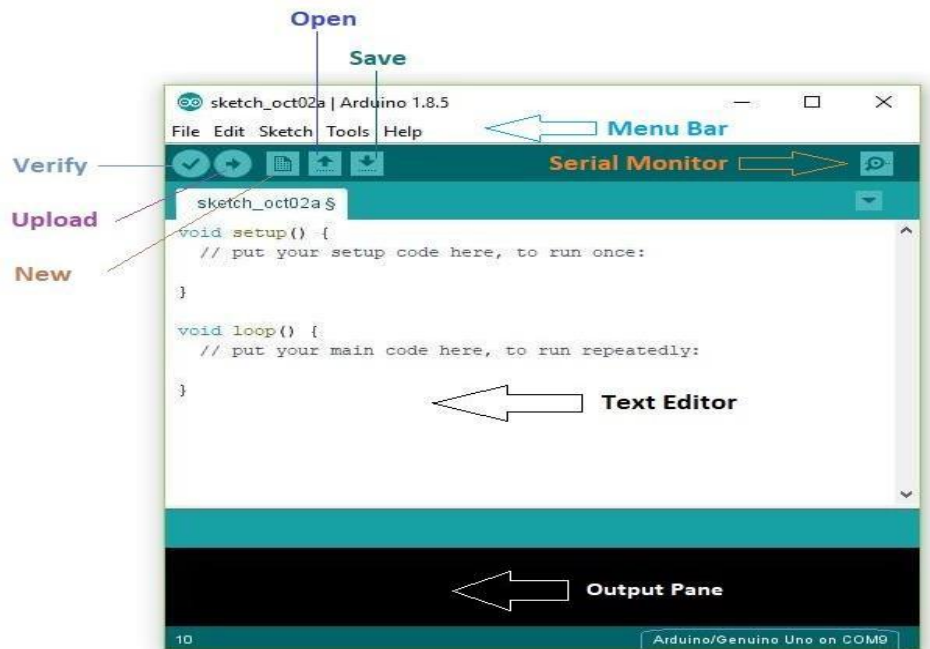
Operating System	:	Windows 10/11
Programming Environment	:	Arduino IDE, Google Colab
IoT Platform	:	Thingspeak IO(for visualizing sensor data)
Libraries	:	Sensor-specific libraries (e.g., for pH, turbidity)
Browser	:	Microsoft Edge

2.2. HARDWARE REQUIREMENTS

Processor	:	AMD RYZEN 3
Processor Speed	:	3.06 GHz
RAM	:	8 GB
Solid State Drive	:	256 GB
Monitor	:	“17” inches

2.3. ABOUT SOFTWARE

Arduino IDE

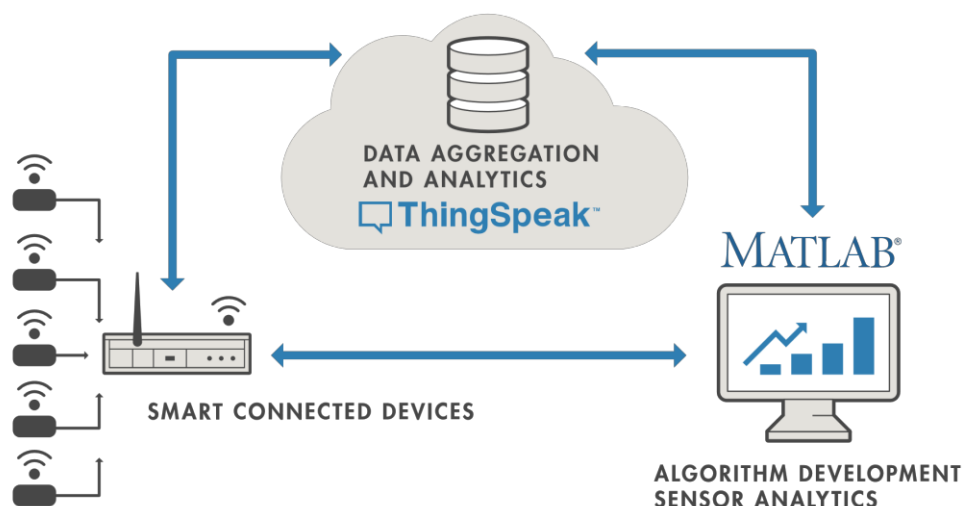


Code for Arduino boards is created using the software tool known as the Arduino Integrated Development Environment (IDE). The Arduino IDE has the following salient qualities and features:

- **User interface:** Users can write, build, and upload code to an Arduino board using the Arduino IDE's user-friendly interface.
- **Code editor:** The integrated development environment (IDE) has a code editor with tools including syntax highlighting, auto-completion, and code folding.
- **Library manager:** This feature of the IDE makes it simple for users to add, remove, and manage libraries for their projects.
- **Serial monitor:** The IDE comes with a serial monitor that enables users to transmit and receive data from an Arduino board over a serial connection.

- **Board manager:** The IDE comes with a board manager that lets users manage and install board definitions for various Arduino boards.
- **Sketchbook:** The IDE comes with a sketchbook where users can store and arrange their code files.
- **Support for several operating systems:** The Arduino IDE is available for Windows, Mac OS X, and Linux.
- **Open-source:** The Arduino IDE is software that, with certain restrictions, allows users to edit and redistribute the code.

ThingSpeak (Cloud)



A platform for Internet of Things (IoT) applications called ThingSpeak is hosted in the cloud. Following are a few of ThingSpeak's salient qualities and traits:

Data gathering: With the help of ThingSpeak, users may gather and store data from IoT gadgets like sensors and actuators in the cloud.

Data sharing: Using APIs (Application Programming Interfaces), ThingSpeak users can openly or privately share their data with other users.

Integration with other platforms: To provide further functionality, ThingSpeak may be integrated with other platforms like MATLAB, IFTTT (If This Then That), and Twitter.

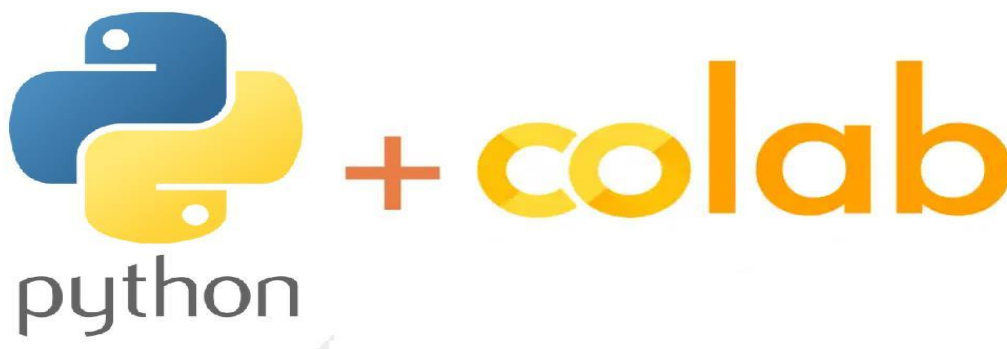
Channels: Channels are containers for the data from a particular device or application, and ThingSpeak groups data into them.

API: ThingSpeak offers an API that enables users to transmit and receive information from their channels as well as carry out additional actions like adding and removing channels.

Widgets: To display real-time data from channels on websites or mobile apps, ThingSpeak offers widgets.

Scalability: ThingSpeak is appropriate for use in smart cities, agriculture, and industrial automation because to its capacity to scale up to handle enormous amounts of data from numerous devices.

Google Colab



A cloud-based platform for coding and data analysis, Google Colab—also called Google Colaboratory—is called Google Colab. The following list of Google Colab's essential qualities and features:

Free to use: The user does not need to install or set up Google Colab on their computer in order to utilise it. Users can use their Google account to access Colab using their web browser.

Editing together: Google Colab enables simultaneous editing by a number of individuals on the same notebook. Users can collaborate on projects and share notebooks with one another.

Support for GPUs and TPUs: Google Colab gives users access to GPUs (Graphics Processing Units) and TPUs (Tensor Processing Units), which can be utilised to speed up computations, especially for applications involving machine learning and deep learning.

Integration with Google Drive: Users can save and access their notes and data files from Drive thanks to Google Colab's integration with Drive.

Pre-installed libraries: NumPy, Pandas, TensorFlow, and PyTorch are just a few of the pre-installed libraries for data analysis, machine learning, and deep learning that come with Google Colab.

Simple installation of additional libraries: Users do not need to install additional libraries on their local workstation in order to install them using conda or pip.

Templates and code snippets: Google Colab offer templates and code snippets for typical activities, making it simple for users to get started with coding and data analysis.

Computing in the cloud: Google Colab utilises the company's cloud infrastructure, which offers high-performance computing capabilities and enables users to run code that might be too computationally complex for their local workstation.

Version control: Git version control is supported by Google Colab, allowing users to follow changes to their code and work together.

Automatic save and backup: Google Colab automatically save and backups notebooks to make sure users don't lose their work in the event of an unanticipated malfunction or outage.

Export options: Users of Google Colab have the ability to export notebooks in a number of different file types, including HTML, PDF, and Python scripts.

3. SYSTEM DESIGN

3.1. MODULES DESCRIPTION

Sensors:

- Sensors collect real-time data related to water quality parameters (e.g., pH, turbidity, temperature, dissolved oxygen, tds).

pH - A pH sensor is an electronic device that gauges a liquid's acidity or alkalinity.

turbidity - A turbidity sensor is an electronic tool that assesses the concentration of suspended particles, such as solids or bacteria, in a liquid.

temperature – A temperature (DS18B20) sensor measures temperature in degrees Celsius with 9 to 12-bit precision

Dissolved oxygen – A DO is defined as the amount of oxygen that is present in water.

Tds - The concentration of dissolved solids in a liquid is measured using an electrical device called a TDS (Total Dissolved Solids) sensor

ESP32:

- The ESP32 is a well-liked system-on-chip (SoC) microcontroller that is frequently used in Internet of Things (IoT) and other embedded systems applications. The ESP32 microcontroller processes the sensor data and transmits it to the cloud for further analysis.

Thingspeak (Cloud):

- ThingSpeak is an IoT analytics platform service that allows you to aggregate, visualize, and analyze live data streams in the cloud. You can send data to ThingSpeak from your devices, create instant visualization of live data, and send alerts.

Model Implementation Using ML:

- Data from the cloud is processed using machine learning models to classify or predict water potability.

Three ML algorithms are mentioned:

- **Support Vector Machine (SVM):** A supervised learning model used for classification tasks.
- **Random Forest:** An ensemble learning method that uses decision trees for improved accuracy.
- **Extreme Gradient Boosting (XGBoost):** A powerful gradient-boosted decision tree algorithm known for high performance.

Condition Check for Potability of Water:

- The output from the ML models is used to determine whether the water meets potability standards.

Validation:

- The results are validated against predefined conditions or thresholds to ensure accuracy.

Display Results on Web App:

- The validated results are displayed on a web application for user access and monitoring.

3.2 FUNCTIONAL REQUIREMENTS

Identify Key Parameters: Finding the important metrics to monitor is the first stage in creating a machine learning-based water quality monitoring system. These could be chemical parameters like nutrient levels, heavy metal concentrations, and organic pollutants, as well as physical parameters like pH, TDS, and turbidity.

Collect Data: Data on these characteristics must be gathered from a variety of sources, including water quality sensors, remote sensing data, and laboratory examination, once the critical parameters have been identified.

Data Pre-processing: Pre-processing is necessary to get rid of outliers, mistakes, and missing values from the acquired data. Data normalisation, data cleaning, and data transformation may all be part of this process.

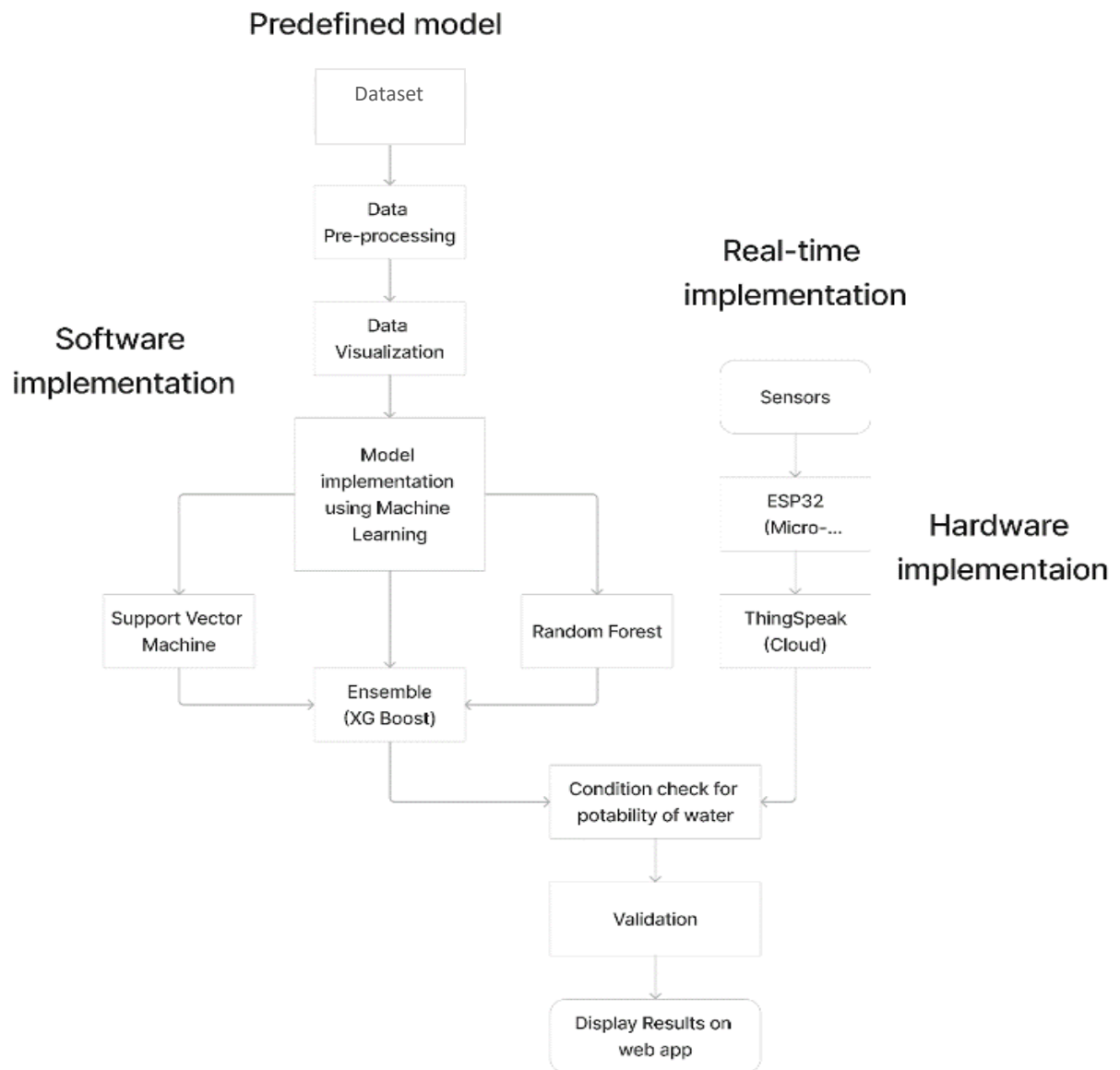
Feature Selection and Extraction: While feature extraction involves turning raw data into useful features that can be used in machine learning models, feature selection entails choosing the parameters that have the greatest influence on water quality.

Machine Learning Model Selection and Development: For the purpose of monitoring water quality, a variety of machine learning models, such as decision trees, random forests, support vector machines, and neural networks, can be utilised. The chosen model should take into account the features of the data, the complexity of the issue, and the desired results. The machine learning algorithm is calibrated to forecast levels of water quality using past data.

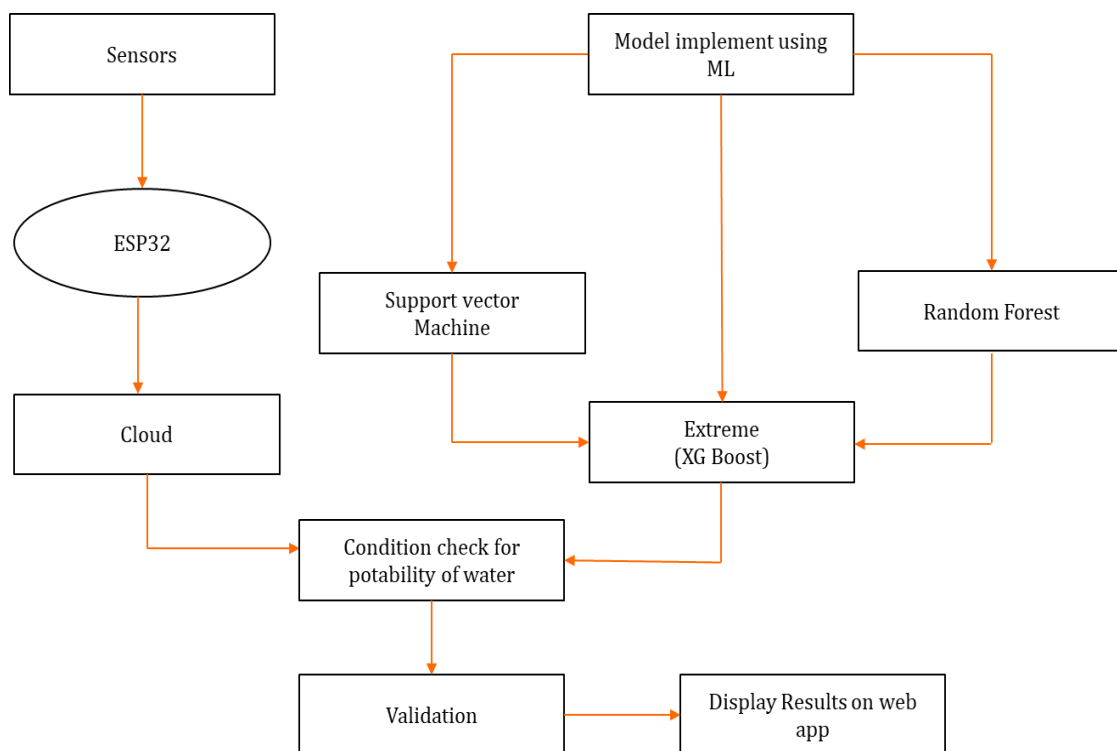
Model Evaluation: The efficiency of the machine learning model is evaluated using metrics including accuracy, precision, recall, and F1 score. To make sure the model can generalise to fresh data, cross-validation techniques can also be used to test it.

Deployment: The created machine learning model can be used as a real-time water quality monitoring system to continually track water quality levels, spot abnormalities, and forecast future water quality levels.

3.2.1. SYSTEM DIAGRAM



3.2.2. DATA FLOW DIAGRAM



3.4. USER INTERFACE DESIGN

The sensors are used by the microcontroller to collect data from the Wastewater. After that, various machine learning techniques were applied to the sensor data.

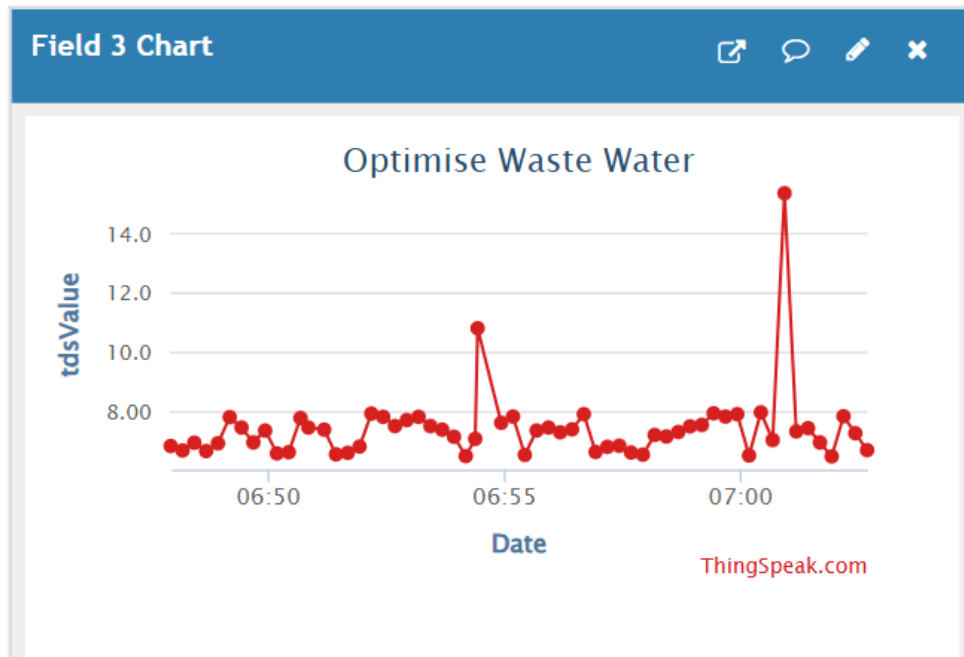
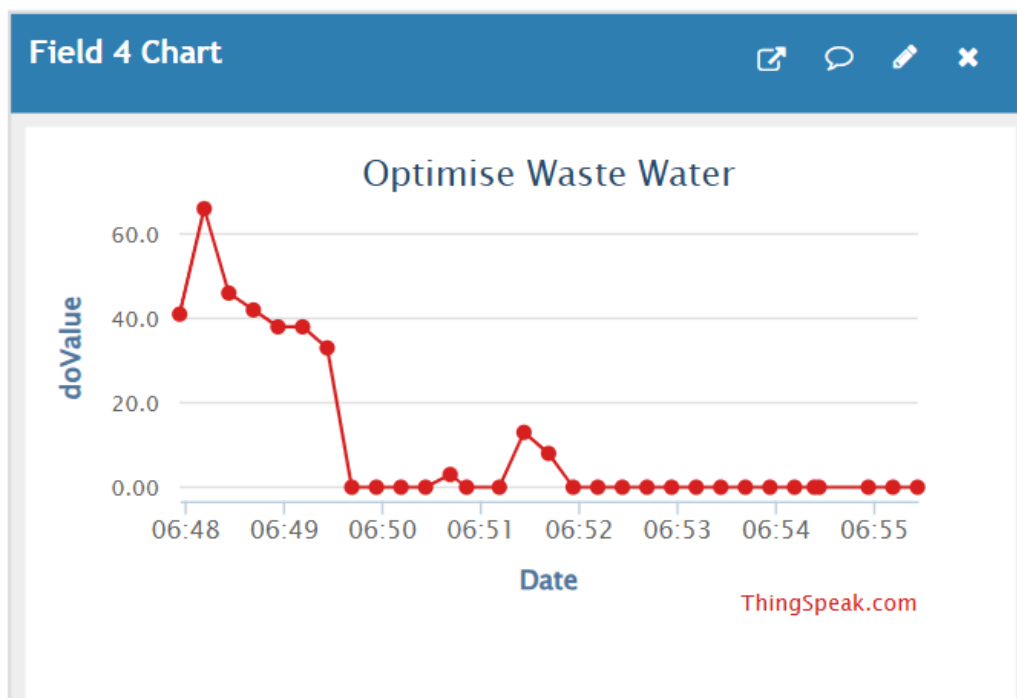
Turbidity Graph



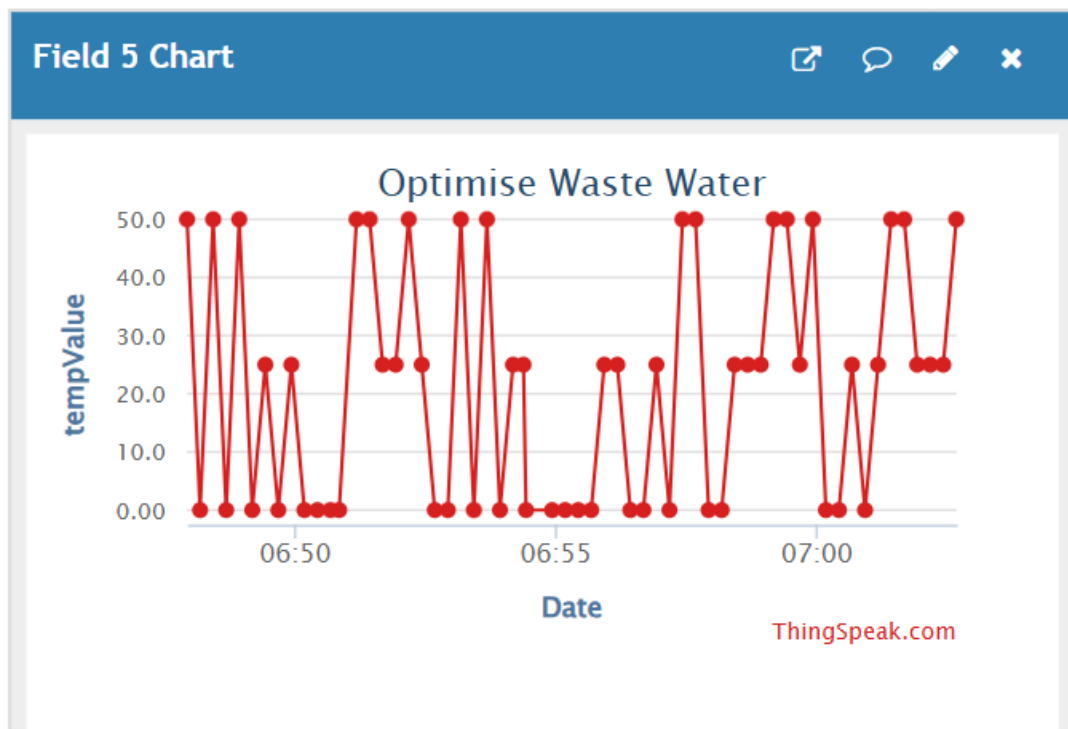
pH Graph



tds Graph

**doValue**

Temperature Graph



4. SYSTEM IMPLEMENTATION

Implementation using ML

Depending on the test results, we may determine whether the Waterwater is Consume or not depending on the values predicted by our proposed model. By importing the pre-trained model and then deploying it, we created a web application for our proposed model that presents real-time data about the water being tested together with potability values.

The methods we employed in calculating the potability values, which are crucial for classifying. By applying the test values to a trained model, this is accomplished. The pre-training of the model was preceded by the data pre-processing, which included the elimination of extraneous data attributes.

As a result, the model was trained exclusively on TDS, turbidity and pH. All null values that occurred in the data attributes were then eliminated.

The accuracy, F1 score and recall values of our proposed model

Evaluation of water quality parameters of based on proposed methodology

Model	Accuracy	F1 Score	Recall
Random Forest	69.35	57.24	57.77
SVM	65.85	58.23	59.61
XG boost	95.12	94.74	94.06

The maximum accuracy of 95.12% is achieved using XG boost. Later for validation of our model, we have used logistic regression, kNN and decision tree.

Model	Accuracy	F1 Score	Recall
Logistic Regression	64.17	40.51	81.98
Decision tree	65.58	52.26	55.55
Random forest	69.35	57.24	57.77
kNN (k = 7)	63.56	56.84	57.05
SVM	65.85	58.23	59.61
XG Boost	95.12	94.74	94.06

the accuracy of SVM, kNN, Random Forest, Decision Tree, Logistic Regression & XG Boost.

The models we utilised had high recall and precision along with F1 score. It demonstrates that of all the approaches, the XG Boost has the highest accuracy.

5. SYSTEM TESTING

Introduction to System Testing

In this section, you would provide an overview of system testing, its importance in the lifecycle of an IoT project, and its role in ensuring that the system functions as expected before deployment.

- Types of Testing
 - Unit testing: Testing individual components like sensors, microcontroller, and display.
 - Integration testing: Testing how the hardware and software components work together.
 - System testing: Testing the entire system's functionality and performance in a real-world scenario.

Unit Testing

- Testing Each Component
 - Sensor Testing: Describe how each sensor is tested independently to ensure they return accurate readings.

Integration Testing

- Hardware Integration
 - Testing the full circuit (ESP32, sensors, and output devices) to ensure proper data flow.
 - Checking the response time and consistency of data from sensors to the ESP32 and display.

- Software Integration
 - Ensuring that the code works seamlessly to integrate the hardware with the cloud services.
 - MQTT Protocol Testing: Verify that the data is being sent to the cloud properly and check for any delays or errors.

System Testing

- Testing System with Real-World Data
 - Conduct tests with actual water quality data in various conditions to verify that the system reacts to changes in air quality
 - Data Logging: Monitor the data sent to the cloud, verify if the system stores the correct data and provides meaningful analytics.
- Security Testing
 - Verifying the security of the data being transmitted via MQTT.
 - Ensuring that the data is securely stored in the cloud and that unauthorized users cannot access the system.
- User Acceptance Testing (UAT)
 - Gathering feedback from potential users (e.g., industrial workers, home users) to verify that the system meets their needs.

Ensuring the system is easy to use, with a user-friendly interface on the display and cloud platform.

6. SYSTEM MAINTENANCE

Introduction to System Maintenance

In this section, you would explain the importance of maintaining an IoT system after it is deployed, ensuring its continued functionality and addressing any issues that arise.

- Categories of Maintenance
 - Corrective Maintenance: Fixing issues that cause the system to malfunction.
 - Preventive Maintenance: Regular checks and updates to ensure the system remains functional.
 - Adaptive Maintenance: Making changes to the system in response to new requirements or environments.

Routine Maintenance Tasks

- Sensor Calibration
 - Over time, sensors may require recalibration. Discuss the need to recalibrate sensors for accurate water quality measurements.
 - Temperature may need recalibration as well, especially in extreme environmental conditions.
- Firmware Updates
 - Periodic updates to the firmware to improve performance, add features, or fix bugs.
 - Discuss how firmware updates are deployed to ESP32 and other components, and their impact on system operation.

- Cloud Maintenance
 - Ensuring that data is being correctly sent to and stored in the cloud.
 - Handling potential issues with cloud connectivity and keeping the cloud platform updated for security and performance.
- Troubleshooting and Error Fixing
 - Identifying common system failures, such as sensor malfunctions, communication failures between ESP32 and cloud, or display issues.
 - Procedures for troubleshooting hardware and software issues, including diagnostic tools and methods.

Long-Term Scalability

- Scaling the System
 - Adding more sensors or connecting multiple IoT devices to monitor a larger area or environment.
 - Ensuring that the system is scalable to meet future needs, such as integrating more advanced sensors or extending cloud storage.

7. Conclusion

Summary of the Project

In this section, you would summarize the entire project, highlighting the key goals, the system's design, implementation, and the results achieved.

- Objectives Achieved
 - Explain how the project successfully monitors water quality, categorizes levels, and optimize of Wastewater .
- Performance Evaluation
 - Assess how well the system performs in terms of accuracy, reliability, and user satisfaction.
 - Discuss any challenges faced during development and how they were overcome.

Implications of the System

- Discuss the significance of this Optimizing Wastewater through AI in improving environmental safety.
 - Potential impact on industrial, residential, and commercial spaces.
 - How this technology can lead to better health outcomes by providing timely alerts to hazardous air quality conditions.

Concluding Remarks

- Wrap up by discussing the importance of IoT in environmental monitoring.

8. FUTURE ENHANCEMENT

Introduction to Future Enhancements

This section would explore ways the current air quality monitoring system could be improved and expanded in the future.

- Possible Upgrades
 - Adding more sensors to monitor additional environmental factors such as particulate matter (PM2.5, PM10), noise pollution, or volatile organic compounds (VOCs).
 - Improving the system's data analytics and providing more detailed insights into the air quality trends.
- Improved User Interfaces
 - Developing a more user-friendly cloud platform interface, mobile apps, or web-based dashboards to visualize air quality data.
 - Integrating voice assistants like Alexa or Google Home for voice notifications.
- Advanced Data Analysis
 - Using machine learning or AI for predictive analytics based on historical data to forecast air quality trends.
 - Implementing automated actions, like turning on air purifiers or fans when poor air quality is detected.

9. BIBLIOGRAPHY

Books

- [1] V. Akkula, Tutorial on support vector machine (SVM), School of EECS, Washington State University, vol. 37, pp-3, 2006
- [2] G. Biau, The Journal of Machine Learning Research, "Analysis of a Random Forests Model," in JMLR. org, vol.13, pp 1063-1095, 2012
- [3] S. Zhang, X. Li, M. Zong, X. Zhu and D. Cheng ACM Transactions on Intelligent Systems and Technology, Learning k for KNN classification (TIST), ACM New York, NY, USA, vol. 8, pp-1-19, 2017
- [4] S. Lemeshow and D. Hosmer, Applications of logistic regression analysis to ophthalmic research in the American Journal of Ophthalmology, published by Elsevier, vol. 147, pp. 766-767, 2009.
- [5] S. Kotsiantis, Recent information about decision trees from Artificial Intelligence Review, in Springer, vol. 39, pp. 261-283, 2013.
- [6] T. Chen, T. He, M. Benesty and V. Khotilovich, Y. Tang, H. Cho, K. Chen, R. Mitchell, I. Cano, T. Zhou and others, Extreme gradient boosting is known as Xgboost., in Microsoft, vol. 1, pp. 1-4, 2015
- [7] T. Aldhyani, M. Al-Yaari, H. Alkahtani, M. Mashael and others, Water quality prediction using artificial intelligence algorithms, in Hindawi, vol. 2020, 2020.
- [8] M. Azrou, J. Mabrouki, G. Fattah, and A. Guezzaz and F. Aziz, Machine learning algorithms for efficient water quality prediction, in Springer, vol. 8, pp. 2793-2801, 2022

10. APPENDIX

10.1. SAMPLE USER INTERFACE SCREEN

Import Function for the data received

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

[ ] train_df = pd.read_csv("water_quality.csv")
train_df
```

Random Forest

```
[ ] from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=100)
rf_model.fit(x_train, y_train)
```

Support Vector Machine

```
[ ] rf_preds = rf_model.predict_proba(x_test)
svm_preds = svm_model.decision_function(x_test)
```

```
[ ] svm_model = SVC(kernel='rbf', C=1.0)
    svm_model.fit(x_train, y_train)
```

Extreme Gradient Boost

```
[ ] import xgboost as xgb
    import numpy as np
    xgb_model = xgb.XGBClassifier()
    xgb_model.fit(np.column_stack((rf_preds[:, 1], svm_preds)), y_test)
```

10.2. SAMPLE SOURCE CODE

Data Sensing Using ESP32 Code

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// WiFi credentials
const char* ssid = "Nothing Phone (2a)"; // Replace with your Wi-Fi SSID
const char* password = "05100204"; // Replace with your Wi-Fi password

// ThingSpeak API details
const char* server = "http://api.thingspeak.com";
String apiKey = "XUF8WTYWJAD06I54"; // Replace with your ThingSpeak API key

// Pin definitions for sensors
#define ONE_WIRE_BUS 4 // DS18B20 temperature sensor pin
#define PH_SENSOR_PIN 34 // Analog pin for pH sensor
#define TURBIDITY_SENSOR_PIN 35 // Analog pin for turbidity sensor
#define TDS_SENSOR_PIN 32 // Analog pin for TDS sensor
#define DO_SENSOR_PIN 33 // Analog pin for DO sensor

// OneWire and DallasTemperature setup for DS18B20
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

void setup() {
  Serial.begin(9600);
  // Initialize Wi-Fi connection
  WiFi.begin(ssid, password);
  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }
  Serial.println("\nConnected to Wi-Fi");

  // Start DS18B20 temperature sensor
  sensors.begin();
}
```

```

void loop() {
  // Read temperature from DS18B20
  sensors.requestTemperatures();
  float temperature = sensors.getTempCByIndex(0);

  // Read analog values from other sensors
  int phValue = analogRead(PH_SENSOR_PIN);
  int turbidityValue = analogRead(TURBIDITY_SENSOR_PIN);
  int tdsValue = analogRead(TDS_SENSOR_PIN);
  int doValue = analogRead(DO_SENSOR_PIN);

  // Convert raw values to meaningful data (calibration required)
  float ph = map(phValue, 0, 4095, 0, 14); // Example mapping for pH range
  float turbidity = map(turbidityValue, 0, 4095, 0, 100); // NTU scale example
  float tds = map(tdsValue, 0, 4095, 0, 1000); // ppm scale example
  float dissolvedOxygen = map(doValue, 0, 4095, 0, 10); // mg/L scale example

  // Print values to Serial Monitor
  Serial.println("Sensor Readings:");
  Serial.print("Temperature: "); Serial.println(temperature);
  Serial.print("pH: "); Serial.println(ph);
  Serial.print("Turbidity: "); Serial.println(turbidity);
  Serial.print("TDS: "); Serial.println(tds);
  Serial.print("Dissolved Oxygen: "); Serial.println(dissolvedOxygen);
  // Send data to ThingSpeak
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    String url = String(server) + "/update?api_key=" + apiKey +
      "&field1=" + String(temperature) +
      "&field2=" + String(ph) +
      "&field3=" + String(turbidity) +
      "&field4=" + String(tds) +
      "&field5=" + String(dissolvedOxygen);

    http.begin(url);
    int httpResponseCode = http.GET();
  }
}

```

```

if (httpResponseCode > 0) {
  Serial.println("Data sent to ThingSpeak successfully");
} else {
  Serial.print("Error sending data: ");
  Serial.println(httpResponseCode);
}
http.end();
} else {
  Serial.println("Wi-Fi disconnected");
  WiFi.begin(ssid, password); // Reconnect if disconnected
  delay(1000);
  return;
}

delay(15000); // Wait before sending the next set of data (ThingSpeak limit: every ~15
seconds)
}

```